

SAP Job Automator — 4-Week Demo-First Execution Plan

Date: March 2026 Goal: Make the product investor-demo ready fast, without risky large refactors.

Strategy

This plan intentionally avoids major rewrites (splitting monoliths, multi-tenant migration, billing, admin panel).

Focus order:

1. Demo reliability
2. UX polish on the hero flow
3. Light safeguards + observability
4. Demo script and assets

Success criterion: you can run a 12–15 minute demo end-to-end without breakage, confusion, or rough edges.

Scope Locks (Do Not Touch in 4 Weeks)

- No full `server.js` decomposition
- No full `js/generate.js` decomposition
- No multi-tenant migration
- No billing system
- No admin dashboard
- No framework migration

Reason: these are high-risk and low-demo-value in current timeline.

Week 1 — Stability + Friction Removal

Objective

Remove the biggest UX breakages and make the generation flow consistently usable.

Tasks

1. Generation flow reliability pass

- Validate queue item lifecycle states in `js/generate.js` (`idle` → `generating` → `done/error`).
- Confirm no state desync after regenerate, DACH fix, or selector changes.

2. CV selector stability

- Keep drawer open during checkbox toggles (already fixed).
- Verify close/confirm/reset behavior across multiple queue items.
- Ensure pinned selections persist per job key in `genState.selectors` .

3. Data hygiene validation

- Ensure no stale references to removed education history in:
 - `data/skill_data_bank.json`
 - `data/vector_store.json`
 - `data/writing_style_profile.json`
- Run text search for disallowed phrases before demo prep.

4. Core endpoint sanity checks

- Validate these endpoints manually: `/health` , `/scrape` , `/fetch-jd` , `/jd-summary` , `/generate` , `/dach-check` , `/export-pdf` .

Acceptance criteria

- Generate queue works for at least 3 jobs in one session without UI glitch.
- CV selector behaves predictably and does not collapse mid-selection.
- No disallowed historical references appear in generated outputs from test prompts.

Week 2 — UX Polish (Investor-visible Layer)

Objective

Make the product feel intentional and premium in the first 90 seconds.

Tasks

1. Generate view visual hierarchy

- Improve readability of fit badges, requirement groups, and document panes.
- Align spacing and card consistency across topbar/cards/results.
- Keep button hierarchy clear: primary vs secondary vs status chip.

2. Search view polish

- Improve result table scannability (column balance, badge readability).
- Add better loading feedback during scrape.

3. Auth first impression

- Replace rotating motivational text with concise product positioning.
- Add cleaner loading state on sign-in action.

4. Empty/loading/error states

- Standardize visible states in Search + Generate views.
- Ensure every long-running action has immediate user feedback.

Acceptance criteria

- A first-time viewer can understand value prop and next action in each view within 10 seconds.
 - No visually broken sections at 1280px width.
 - Generate view looks cohesive (cards, spacing, typography, badges, controls).
-

Week 3 — Lightweight Hardening + Observability

Objective

Add minimal but meaningful safeguards without a full security refactor.

Tasks (minimal set)

1. CORS restriction

- Restrict origin in `server.js` via `ALLOWED_ORIGIN` env fallback.

2. Safe error responses

- Return generic error messages in production mode.
- Keep detailed messages only in development.

3. Request logging

- Add basic structured request log middleware for traceability.

4. Manual abuse guard (lightweight)

- Add a simple in-memory limiter for high-cost endpoints (`/generate` , `/export-pdf` , `/dach-fix`).

Acceptance criteria

- Server logs each request with status/latency.
- Demo cannot accidentally leak internal stack/file info in production mode.
- Repeated spam calls are throttled at basic level.

Week 4 — Demo Packaging + Proof of Reliability

Objective

Package the product for investor conversations.

Tasks

1. Smoke test script

- Add simple script covering:
 - `/health`
 - `/jd-summary`
 - `/generate` (cv)
 - `/dach-check`
 - `/export-pdf`

2. Demo script (15 mins)

- Finalized click-path with expected outcomes and fallback path if one step fails.

3. Investor-facing one-pager

- Problem
- Product workflow
- Differentiators (RAG + style profile + DACH compliance)
- Why now
- What next

4. Final rehearsal checklist

- Environment ready (`.env` , Firebase, pdflatex, Playwright)
- 2–3 preselected job URLs
- One backup generated output in case network/API slowdown

Acceptance criteria

- Two consecutive full demo rehearsals succeed with no blocking errors.
- You can explain architecture and moat in under 3 minutes with confidence.

Weekly Deliverables Snapshot

- **End of Week 1:** Stable generation + selector + clean data behavior
 - **End of Week 2:** Premium UX pass on Search/Generate/Auth
 - **End of Week 3:** Basic safeguards + logs
 - **End of Week 4:** Demo assets, smoke tests, investor-ready narrative
-

KPI Targets for Demo Readiness

- Generation success rate in local rehearsal: $\geq 90\%$
 - End-to-end demo completion time: ≤ 15 min
 - UI-critical glitches per rehearsal: 0
 - Output quality defects requiring manual major rewrite during demo: ≤ 1
-

Suggested Next File (Optional)

After this plan, create:

`INVESTOR_DEMO_RUNBOOK.md`

with:

- exact click sequence
- expected screen states
- narration script per step
- fallback script if an API/provider fails

This prevents panic during live demo and keeps the story tight.